

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: ГЕНЕТИЧЕСКИЕ АЛГОРИТМЫ

Студенты гр.
4345

М.А. Гавриш
А.Д. Шувалов
А. Трушкин

Руководитель

Т.Р. Жангиров

Санкт-Петербург
2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студенты: М.А. Гавриш, А.Д. Шувалов, А. Трушкин

Группа: 4345

Тема практики: Генетические алгоритмы

Задание на практику:

Разработать программу с графическим пользовательским интерфейсом для решения задачи аппроксимации полиномиальной функции $f(x)$ степени не выше 8 ступенчатой функцией $g(x)$ с заданным количеством ступеней на интервале $[l, r]$.

Сроки прохождения практики: 06.07.2026 – 18.06.2026

Дата сдачи отчета: 17.06.2026

Дата защиты отчета: 17.06.2026

Студент	_____	М.А. Гавриш
Студент	_____	А.Д. Шувалов
Студент	_____	А. Трушкин
Руководитель	_____	Т.Р. Жангиров

АННОТАЦИЯ

В ходе учебной практики разработана программа для аппроксимации полиномиальной функции ступенчатой функцией методом генетического алгоритма. Реализованы модули для корректных математических вычислений, разработан графический интерфейс пользователя на базе библиотеки PySide6. Поддерживаются различные способы ввода параметров задачи, встроена система пошаговой визуализации с помощью библиотеки Matplotlib.

SUMMARY

During the practical training, a program was developed to approximate a polynomial function with a step function using a genetic algorithm. Modules for accurate mathematical calculations were implemented, and a graphical user interface was created using the PySide6 library. The program supports various methods for inputting problem parameters and includes a step-by-step visualization system based on the Matplotlib library.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Предметная область	6
1.2 Задачи практики	6
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	8
2.1. Изучение теоретических основ.....	8
2.2. Изучение методов аппроксимации.....	8
2.3. Составление плана	8
2.3.1. Представление данных.....	8
2.3.2. Функция качества	8
2.3.3. Общий алгоритм	8
2.3.4. Прототип графического интерфейса будущей программы.....	9
2.3.5. Модуль интеграции и работы с данными	9
2.4. Генетический алгоритм	10
2.4.1. Функция приспособленности	11
2.4.2. Основные составляющие алгоритма	11
2.5. GUI.....	12
2.5.1. Главное окно	12
2.5.2. Виджеты и объекты.....	13
2.5.3. Отрисовка графиков	15
2.6. Управление данными и конфигурацией алгоритма	16
2.6.1. Класс GAConfig	16
2.6.2. Класс DataManager	17
2.6.3. Контроллер приложения.....	17
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	20
4 ОТЗЫВ РУКОВОДИТЕЛЯ	21
ЗАКЛЮЧЕНИЕ	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24
ПРИЛОЖЕНИЕ А	25

ВВЕДЕНИЕ

Генетические алгоритмы представляют собой эвристические методы оптимизации, основанные на принципах естественного отбора и генетики. Данные алгоритмы находят широкое применение в задачах, где традиционные методы оптимизации оказываются неэффективными или неприменимыми ввиду высокой размерности пространства решений, нелинейности целевой функции или отсутствия аналитического решения.

Предметной областью данной практики является разработка программы для численной аппроксимации функций с использованием эволюционных вычислений. Использование генетических алгоритмов для решения данной задачи позволяет находить приемлемые решения в условиях оптимизации параметров аппроксимирующей функции.

Целью практики является разработка программы с графическим интерфейсом, реализующего генетический алгоритм для минимизации расстояния между полиномиальной функцией и ступенчатой функцией заданной структуры.

Для достижения поставленной цели необходимо решить следующие задачи: изучить теоретические основы генетических алгоритмов и методов аппроксимации функций, разработать структуру данных для представления полиномов и ступенчатых функций, реализовать операторы генетического алгоритма (селекцию, скрещивание, мутацию), создать функцию качества для оценки приближения, разработать графический интерфейс с возможностью визуализации процесса поиска решения, обеспечить возможность настройки параметров алгоритма и ввода данных.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Задача приближения полинома ступенчатой функцией формулируется как задача минимизации расстояния между функциями. Параметры оптимизации — высоты ступеней ступенчатой функции при фиксированных границах разбиения интервала. Данная задача является задачей непрерывной оптимизации.

Практическая значимость задачи обусловлена необходимостью разработки инструментов для визуального анализа процесса аппроксимации, что важно для исследования свойств генетических алгоритмов. Разработка программы с пошаговой визуализацией позволяет изучить разницу между популяциями.

1.2 Задачи практики

1. Изучить теоретические основы генетических алгоритмов (селекция, скрещивание, мутация). Ответственные: Гавриш Матвей, Трушкин Александр, Шувалов Алексей.

2. Изучить методы аппроксимации функций, метрики расстояния между функциями. Ответственные: Гавриш Матвей, Трушкин Александр, Шувалов Алексей.

3. Разработать структуру данных для представления полиномиальной функции с заданным интервалом определения. Ответственный – Шувалов Алексей.

4. Разработать структуру данных для представления ступенчатой функции с фиксированными границами ступеней. Ответственный – Шувалов Алексей.

5. Реализовать функцию качества для оценки степени приближения ступенчатой функции к полиному на заданном интервале. Ответственный – Шувалов Алексей.

6. Реализовать основные операторы ГА: инициализацию популяции, селекцию, скрещивание, мутацию и формирование нового поколения. Ответственный – Шувалов Алексей.
7. Разработать механизм сохранения истории поколений для обеспечения возможности возврата к предыдущим состояниям алгоритма. Ответственный – Гавриш Матвей.
8. Спроектировать и реализовать графический интерфейс с возможностью управления параметрами алгоритма. Ответственные: Трушкин Александр, Гавриш Матвей.
9. Реализовать возможность ввода данных разными способами: из файла и путём случайной генерации. Ответственный – Гавриш Матвей.
10. Обеспечить пошаговую визуализацию процесса эволюции и перехода к конечному решению. Ответственный – Трушкин Александр.

2 РЕЗУЛЬТАТЫ РАБОТЫ

Разделение ролей в бригаде:

1. Шувалов Алексей – разработка генетического алгоритма.
2. Трушкин Александр – разработка графического интерфейса.
3. Гавриш Матвей – разработка связи между ГА и GUI.

2.1. Изучение теоретических основ

Были изучены теоретические основы генетических алгоритмов: основные операторы и свойства [1].

2.2. Изучение методов аппроксимации

Был изучен метод аппроксимации с помощью генетического алгоритма.

2.3. Составление плана

Был составлен план по решению задачи:

2.3.1. Представление данных

Целевая функция $f(x)$: используется массив целых чисел для хранения коэффициентов функции, где индекс элемента соответствует степени x .

Ген: вещественное число, равное высоте одной ступени.

Генотип: одномерный массив вещественных чисел, длина которого равна количеству ступеней. Каждый генотип является полноценной ступенчатой функцией.

Популяция: двумерный массив, состоящий из N генотипов.

2.3.2. Функция качества

Расчёт ошибки: $Error = \frac{1}{N} \sum_{i=0}^N |f(x_i) - g(x_i)|$

Приспособленность: $Fitness = \frac{1}{1+Error}$

2.3.3. Общий алгоритм

1. Ввод данных: пользователь программы вводит коэффициенты полиномиальной функции, интервал, количество ступеней и параметры генетического алгоритма (размер популяции, число поколений и шанс мутации). Доступен ручной ввод, случайная генерация данных, чтение из файла.

2. Генерация начальной популяции

3. Цикл эволюции

- Оценка приспособленности для каждой особи в популяции.
- Сохранение особи с наивысшей приспособленностью.
- До тех пор, пока не заполнится новая популяция:
 - Турнир для выявления родителей.
 - Скрещивание.
 - Мутация потомков.
 - Добавление потомков в новую популяцию.
 - Замена старой популяции на новую.

4. Вывод результата: генотип с наивысшей приспособленностью содержит искомые высоты ступеней.

2.3.4. Прототип графического интерфейса будущей программы

Прототип изображен на рис. 1.

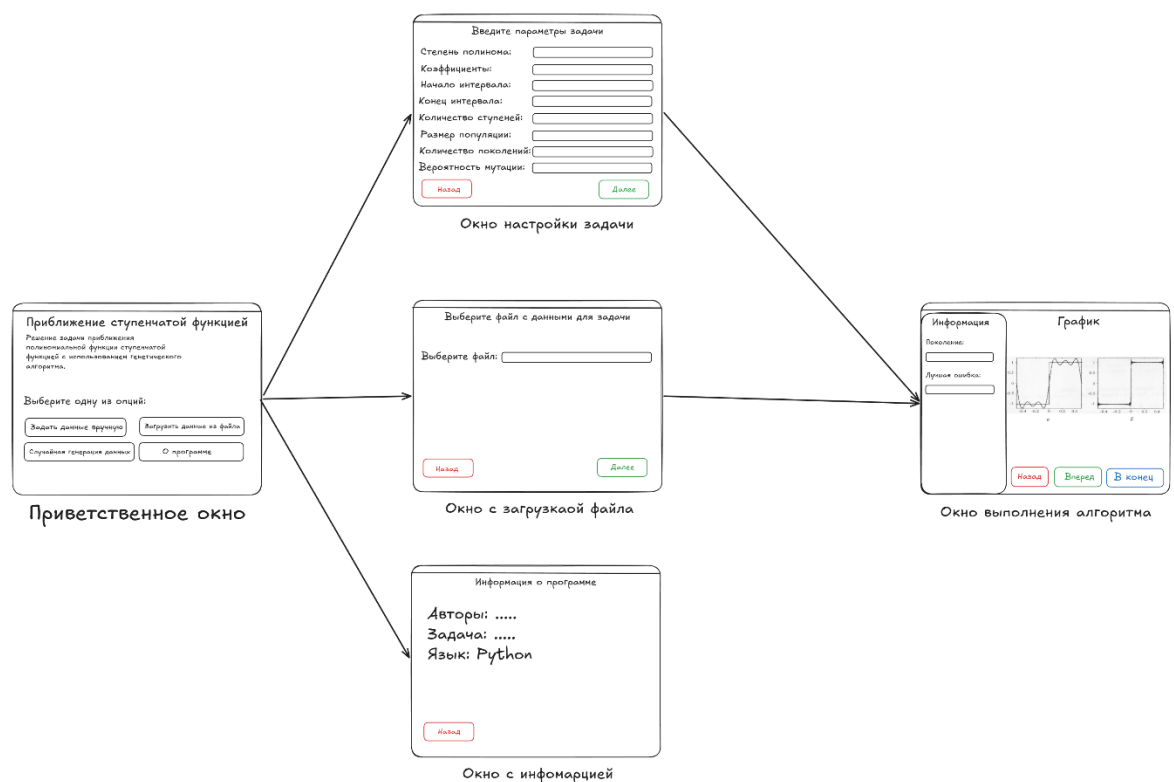


Рисунок 1 — Прототип графического интерфейса

2.3.5. Модуль интеграции и работы с данными

Данный модуль выполняет роль контроллера в архитектуре приложения. Он обеспечивает связь между графическим интерфейсом (gui-prototype) и вычислительным ядром (backend-prototype). Основные задачи модуля:

- Нормализация входных данных
- Чтение/запись параметров задачи из файлов
- Случайная генерация тестовых полиномов
- Управление циклом алгоритма (пошаговый запуск, переход вперёд/назад, пропуск всех промежуточных выводов)

- Хранение истории состояний для визуализации

Логика случайной генерации данных:

- Для режима случайная генерация будет реализован алгоритм создания верных тестовых данных:

- Степень полинома: случайно выбирается от 1 до 8.
- Коэффициенты: генерируются случайные целые числа в диапазоне от -5 до 5.
- Интервал: случайно выбирается в диапазоне от -10 до 10 при условии $l < r$.
- Количество ступеней: случайно выбирается от 10 до 35.

Для реализации требования о возможности вернуться на несколько шагов назад модуль хранит полную историю эволюции. При каждом шаге алгоритма бэкенд возвращает новую популяцию, и контроллер сохраняет её в `self.generations`. При нажатии кнопок «Назад»/«Вперед» в GUI контроллер не пересчитывает алгоритм, а просто меняет `current_step` и отдает GUI сохраненный снимок для отрисовки.

2.4. Генетический алгоритм

Генетический алгоритм предназначен для приближения заданной полиномиальной функции $f(x)$ ступенчатой функцией $g(x)$ на заданном интервале

$[l, r]$. Цель алгоритма – подбор параметров ступеней, при которых сумма разностей между высотами графиков функций будет минимальной. Для хранения коэффициентов исходной функции используется массив целых чисел.

Генотип хранится в виде массива вещественных чисел, длина которого равняется количеству ступеней. Каждый ген в генотипе представляет собой высоту соответствующей ступени. Стартовые значения генов генерируются случайно в пределах от минимального до максимального значения исходной полиномиальной функции.

2.4.1. Функция приспособленности

Оценка генотипа происходит путём суммирования разниц между значениями функций в определенных точках. Данные точки определяются путём разбиения отрезка $[l, r]$ на равные отрезки.

Значение приспособленности вычисляется путем инверсии ошибки. Приспособленность будет стремиться к единице, так как при совпадении графиков ошибка будет равняться нулю.

Для имитации эволюции в данном алгоритме используются: турнирный отбор для трёх генотипов, одноточечное скрещивание и мутация с заданными вероятностями.

2.4.2. Основные составляющие алгоритма

Класс `Polynom` – предназначен для вычисления значения полиномиальной функции в точке и для вычисления её верхней и нижней границ.

Класс `StepFunc` – математическая модель ступенчатой функции. Позволяет вычислить значение функции в точке.

Метод `init_population` – создаёт начальную популяцию случайных особей, распределяя значения генов в вычисленных границах.

Метод `calc_fitness` – вычисляет значения приспособленности для разных генотипов.

Метод `tournament_selection` – выбирает генотип с наивысшей приспособленностью из трёх кандидатов в рамках турнирного отбора.

Метод `crossover` – с заданной вероятностью проводит одноточечное скрещивание двух генотипов для создания новых потомков.

Метод `mutation` – с заданной вероятностью проводит мутацию одного гена в генотипе в пределах 10%.

Метод `generation_step` – осуществляет один шаг эволюции. Сохраняет особь предыдущего поколения с наивысшей приспособленностью, отбирает родителей, скрещивает их, мутирует потомков и формирует новую популяцию.

Метод `save_generation_state` – сохраняет текущее поколение и его метрики на каждом шаге в список словарей.

2.5. GUI

Для разработки графического интерфейса была выбрана экосистема Qt.

- Библиотека GUI: PySide6 [2].
- Инструмент визуального проектирования: Qt Designer – использовался для создания разметки, размещения элементов и управления их свойствами без написания интерфейса вручную [3].
- Утилита компиляции: `pyside6-uic` – применялась для автоматической конвертации визуального файла разметки `welcome.ui` в исполняемый Python-код `welcome_ui.py`.

2.5.1. Главное окно

Основой приложения является класс `MainWindow`.

Разрешение окна зафиксировано на 800×600 пикселей, что гарантирует стабильное отображение интерфейса без искажений на любых мониторах.

Многостраничность: реализована с помощью контейнера `QStackedWidget` (`self.ui.stackedWidget`). Интерфейс разделен на изолированные экраны, переключение между которыми происходит по индексам в коде `setCurrentIndex()`.

Архитектура страниц в проекте:

1. `welcomePage` (Индекс 0): приветственный экран. Содержит название программы, краткое описание и блок главных кнопок для выбора режима работы (рис. 2).
2. `aboutPage` (Индекс 1): страница с общей информацией о программе. Содержит текстовое описание математической постановки задачи аппроксимации и кнопку `backButton_2` для возврата в главное меню.
3. `startPage` (Индекс 2): страница ручного ввода параметров задачи. Именно здесь пользователь настраивает математические свойства для аппроксимации: интервал, степень полинома, коэффициенты и количество ступеней (рис. 3).
4. `paramsPage` (Индекс 3): страница настройки параметров генетического алгоритма. Содержит элементы управления для конфигурирования популяции и операторов генетического поиска (рис. 4).
5. `algorithmPage` (Индекс 4): страница непосредственного выполнения расчетов и визуализации результатов в реальном времени (рис. 5 и рис. 6).

2.5.2. Виджеты и объекты

Внутри страниц были использованы и настроены следующие типы объектов `PySide6`:

- `QFrame` (`mainCard`, `mainCard_2`, `mainCard_3`): специальные контейнеры с белым фоном, внутри которых группируется весь контент страниц. Они отделяют рабочую зону от общего серого фона приложения.
- `QLabel`: использовались для всех текстовых блоков:
 - Заголовки и описания: `titleLabel`, `descriptionLabel`, `descriptionParamLabel`.
 - Подписи к полям ввода: `stepsLabel` (Количество ступеней), `beginningIntLabel` (Начало интервала) и др.
- `QPushButton`: Кнопки управления. Все кнопки имеют фиксированный минимальный размер 280×50 пикселей:

- startButton – кнопка «Ввести данные вручную» на главном экране.
- aboutButton – кнопка «О программе».
- backButton_1, backButton_2, backButton_4, backButton_5 – кнопки «Назад» для обеспечения возможности пошагового возврата на любой из предыдущих этапов работы.
- nextButton_1, nextButton_2, nextButton_3, nextButton_4 – кнопки навигации «Вперед» и запуска алгоритма.
- inEndButton – специализированная кнопка «В конец» для расчетов финального поколения.
- QSpinBox: Поля для ввода числовых параметров.
 - stepsSpinBox – задание количества дискретных ступеней.
 - degSpinBox – ввод степени целевого полинома.
 - sizePopSpinBox – определение численности популяции ГА.
 - numGenSpinBox – определение лимита поколений алгоритма.
- QDoubleSpinBox: Поля для высокоточного ввода вещественных чисел с плавающей запятой:
 - beginnigIntDoubleSpinBox и endIntDoubleSpinBox – границы интервала аппроксимации.
 - mutProbDoubleSpinBox и crossProbDoubleSpinBox – определение вероятностей операторов мутации и скрещивания в диапазоне от 0.0 до 1.0.
- QLineEdit (coefLineEdit): Однострочное текстовое поле, используемое для ввода нескольких коэффициентов полинома строкой через пробел.
- QTabWidget (tabWidget): Двухстраничная панель вкладок, обеспечивающая эргономичное разделение выводимой графической информации

- QWidget (errorPlotContainer, functionPlotContainer): Специализированные пустые виджеты-якоря, выступающие в роли родительских контейнеров для интеграции динамических холстов Matplotlib.

2.5.3. Отрисовка графиков

Для встраивания графической информации непосредственно в форму Qt-приложения используется объектный бэкенд библиотеки Matplotlib, ориентированный на интеграцию с Qt5/Qt6-виджетами [4].

1. Инициализация холстов (init_plots):

Для каждого графика динамически создаются объекты подложки Figure и связующие холсты FigureCanvas. Локальное размещение графиков внутри соответствующих виджетов-якорей реализуется через менеджера компоновки QVBoxLayout, который растягивает холсты на всю доступную область контейнеров.

2. Отрисовка графиков (update_display):

Интерфейс визуализации обновляется централизованно через метод update_display(), который извлекает состояние текущей эпохи из истории работы генетического алгоритма и перерисовывает графики:

График аппроксимации (update_function_plot):

- Холст полностью очищается методом self.function_figure.clear().
- Создается новая координатная сетка add_subplot().
- Производится дискретизация интервала аппроксимации [l, r] на 1000 расчетных точек.
- Для каждой точки вычисляется значение целевого полинома $f(x)$ и значение аппроксимирующей ступенчатой функции $g(x)$, построенной на основе вещественного генотипа выбранной особи (self.current_individual).
- Обе кривые наносятся на график непрерывными линиями через стандартный метод ax.plot(), целевой полином — синим цветом, ступенчатая аппроксимация — красным. На график добавляются легенда, заголовок с указанием номеров текущего поколения и индивида, а также координатная сетка.

- Изменения визуализируются вызовом `self.function_canvas.draw()`. График изменения приспособленности (`update_error_plot`):
- График строится по всей накопленной истории алгоритма.
- По оси абсцисс откладываются поколения, по оси ординат — значения фитнес-функции (приспособленности).
- Строятся две кривые: максимальная приспособленность в поколении (`max_fitnesses`, зеленый цвет) и средняя приспособленность по популяции (`avg_fitnesses`, фиолетовый цвет).

Вызов `self.error_canvas.draw()` обновляет холст на вкладке «Ход оптимизации».

2.6. Управление данными и конфигурацией алгоритма

Для возможности взаимодействия с графическим интерфейсом был разработан модуль управления данными и конфигурацией. Данный модуль отвечает за генерацию задач, сохранение и загрузку параметров из файлов, валидацию входных данных, а также связывание графического интерфейса с вычислительным ядром генетического алгоритма.

2.6.1. Класс `GAConfig`

Класс `GAConfig` предназначен для хранения и валидации параметров генетического алгоритма. Все параметры вынесены в единый конфигурационный объект, что позволяет легко передавать настройки между модулями программы и изменять их через графический интерфейс.

Атрибуты класса:

- `population_size` — размер популяции
- `generations` — максимальное количество поколений
- `mutation_chance` — вероятность мутации одного гена
- `crossover_chance` — вероятность скрещивания двух особей

Метод `validate` — производит проверку параметров алгоритма. Если пользователь задаёт значение, выходящее за допустимые границы, оно автоматически приводится к ближайшему допустимому значению. Это предотвращает некорректную работу алгоритма при ошибочном вводе.

2.6.2. Класс `DataManager`

Класс `DataManager` реализует функционал работы с входными и выходными данными программы. Он обеспечивает три способа получения параметров задачи: ручной ввод через графический интерфейс, загрузку из файла и случайную генерацию.

Метод `generate_random_polynom` — создаёт случайный полином заданной степени со случайными коэффициентами в заданном диапазоне.

Метод `generate_random_task` — формирует полный набор параметров задачи, включая случайный полином, интервал аппроксимации и случайное количество ступеней. Возвращает словарь с параметрами, готовый для передачи в генетический алгоритм.

Метод `save_to_file` — сохраняет параметры задачи и конфигурацию генетического алгоритма в файл формата JSON. Есть возможность сохранять только параметров задачи, так и совместное сохранение с настройками алгоритма.

Метод `load_from_file` — загружает параметры задачи из JSON-файла. Метод возвращает кортеж из словаря с параметрами задачи и объекта конфигурации.

Метод `validate_task_data` — метод валидации загруженных данных. Проверяет наличие всех обязательных полей и их значения. При обнаружении ошибок выбрасывает исключение `ValueError`.

2.6.3. Контроллер приложения

Класс `AppController` связывает графический интерфейс и вычислительные модули программы.

Метод `connect_signals` — устанавливает связь между элементами GUI и методами обработки событий.

Метод `on_manual_input` – обработчик кнопки ручного ввода данных. Запускает процесс перехода к окну ввода параметров полинома, интервала и количества ступеней.

Метод `on_load_file` – обработчик кнопки загрузки данных из файла. Открывает стандартный диалог выбора файла, вызывает метод загрузки `DataManager` и обрабатывает возможные ошибки. При успешной загрузке сохраняет параметры в контроллере для последующего использования алгоритмом.

Метод `on_random_generate` – обработчик кнопки случайной генерации данных. Вызывает метод генерации `DataManager` и сохраняет результат для последующей передачи в генетический алгоритм.

Метод `on_about` – обработчик кнопки «О программе».

Методы `on_back_to_menu`, `on_back_to_input`, `on_back_to_params` – позволяют пользователю переключаться на предыдущие экраны приложения.

Метод `submit_entered_data` – считывает данные, введенные пользователем вручную. Проводит строгую проверку, при успехе формирует основную структуру задачи и выполняет переход на экран настройки алгоритма.

Метод `submit_algorithm_params` – метод, используемый для настройки параметров алгоритма. После успешного ввода значений переходит к методу `run_algorithm`.

Метод `run_algorithm` – создает математическую модель полинома, создает экземпляр класса `GeneticAlgorithm`, генерирует нулевое поколение.

Методы `on_next_generation`, `on_prev_generation` – выполняют пошаговый переход между поколениями вперед и назад.

Метод `go_to_end` – мгновенно производит все итерации генетического алгоритма.

Метод `on_go_to_generation` – позволяет перейти к произвольному поколению по его номеру.

Метод `on_go_to_individual` – позволяет перейти к произвольному индивиду по его номеру.

Метод `run` – запускает главное окно приложения и передаёт управление циклу обработки событий графического интерфейса.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе выполнения учебной практики были использованы следующие технологии и инструменты:

- Язык программирования Python 3.11.
- Библиотека PySide6 — привязка языка Python к инструментарию Qt, совместимая на уровне API с PyQt. Используется для разработки графического пользовательского интерфейса.
- Qt Designer — визуальный редактор интерфейсов из состава Qt, использованный для проектирования разметки окон программы. Результатом работы в Qt Designer является файл формата .ui, который затем конвертируется в Python-код.
- Утилита pyside6-uic. Утилита командной строки pyside6-uic применяется для автоматической конвертации файлов разметки .ui в исполняемый Python-код.
- Формат JSON использован для хранения и обмена данными между модулями программы. Используется для сохранения конфигурации и параметров задачи.
- Git использован для совместной разработки проекта [5].

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на ???

ЗАКЛЮЧЕНИЕ

В ходе учебной практики была разработана программа с GUI для решения задачи аппроксимации полиномиальной функции ступенчатой функцией с использованием генетического алгоритма.

По результатам работы были решены следующие задачи:

1. Изучены теоретические основы генетических алгоритмов, включая операторы селекции, скрещивания и мутации.
2. Изучены методы численной аппроксимации функций.
3. Разработана структура данных для представления полиномиальной функции в виде класса `Polynom` с методами вычисления значения в точке и определения границ функции на заданном интервале.
4. Разработана структура данных для представления ступенчатой функции в виде класса `StepFunc` с границами ступеней и высотами.
5. Реализована функция качества для оценки степени приближения ступенчатой функции к полиному на заданном интервале на основе среднеарифметического модуля разности значений функций.
6. Реализованы основные операторы генетического алгоритма: инициализация популяции, турнирный отбор, одноточечное скрещивание, мутация с ограничением диапазона и формирование нового поколения с сохранением особи с наивысшей приспособленностью.
7. Разработан механизм сохранения истории поколений, для навигации между состояниями алгоритма без необходимости пересчёта.
8. Спроектирован и реализован графический интерфейс с экранами приветствия, ввода данных, настройки параметров алгоритма и визуализации результатов.
9. Реализованы три способа ввода данных: ручной ввод через GUI, загрузка из JSON-файла и случайная генерация.
10. Обеспечена пошаговая визуализация процесса эволюции с возможностью навигации вперёд/назад по поколениям, перехода к конкретному

поколению или индивиду, а также пропуска всех промежуточных выводов до конечного решения.

Разработанная программа демонстрирует работоспособность генетического алгоритма для задачи аппроксимации функций.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Голдберг, Д. Э. Генетические алгоритмы в поиске, оптимизации и машинном обучении / Д. Э. Голдберг. — М.: Вильямс, 2007. — 328 с.
2. Документация PySide6 [Электронный ресурс]. — URL: <https://doc.qt.io/qtforpython/> (дата обращения: 10.07.2026).
3. Qt Designer Documentation [Электронный ресурс]. — URL: <https://doc.qt.io/qt-6/qtdesigner-manual.html> (дата обращения: 10.07.2026).
4. Документация Matplotlib [Электронный ресурс]. — URL: <https://matplotlib.org/> (дата обращения: 10.07.2026).
5. Репозиторий для выполнения учебной практики // URL: https://github.com/AlekseyShuvalov/Practice_26 (дата обращения: 10.07.2026).

ПРИЛОЖЕНИЕ А

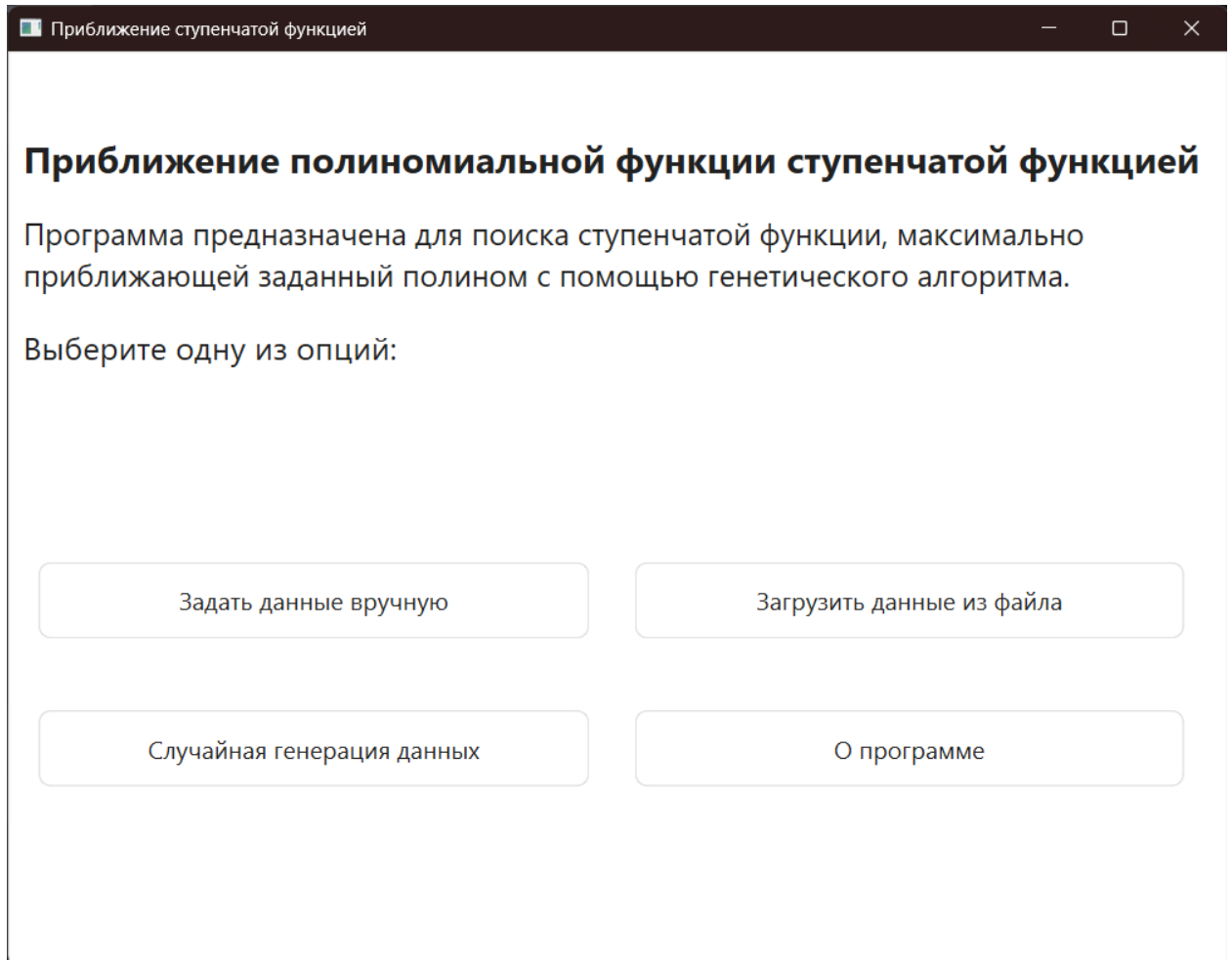


Рисунок 2 — Приветственный экран программы

Приближение ступенчатой функцией

Введите параметры задачи:

Степень полинома: 5

Коэффициенты: 1.5 4 2.28 -0.14 -0.48 1

Начало интервала: -3

Конец интервала: 8

Количество ступеней: 29

Назад Вперед

Рисунок 3 — Окно ввода параметров задачи

Приближение ступенчатой функцией

— □ ×

Введите параметры алгоритма:

Размер популяции	100	▲ ▼
Количество поколений:	250	▲ ▼
Вероятность мутации:	0,20	▲ ▼
Вероятность пересечения:	0,80	▲ ▼

Назад

Вперед

Рисунок 4 — Окно настройки параметров алгоритма

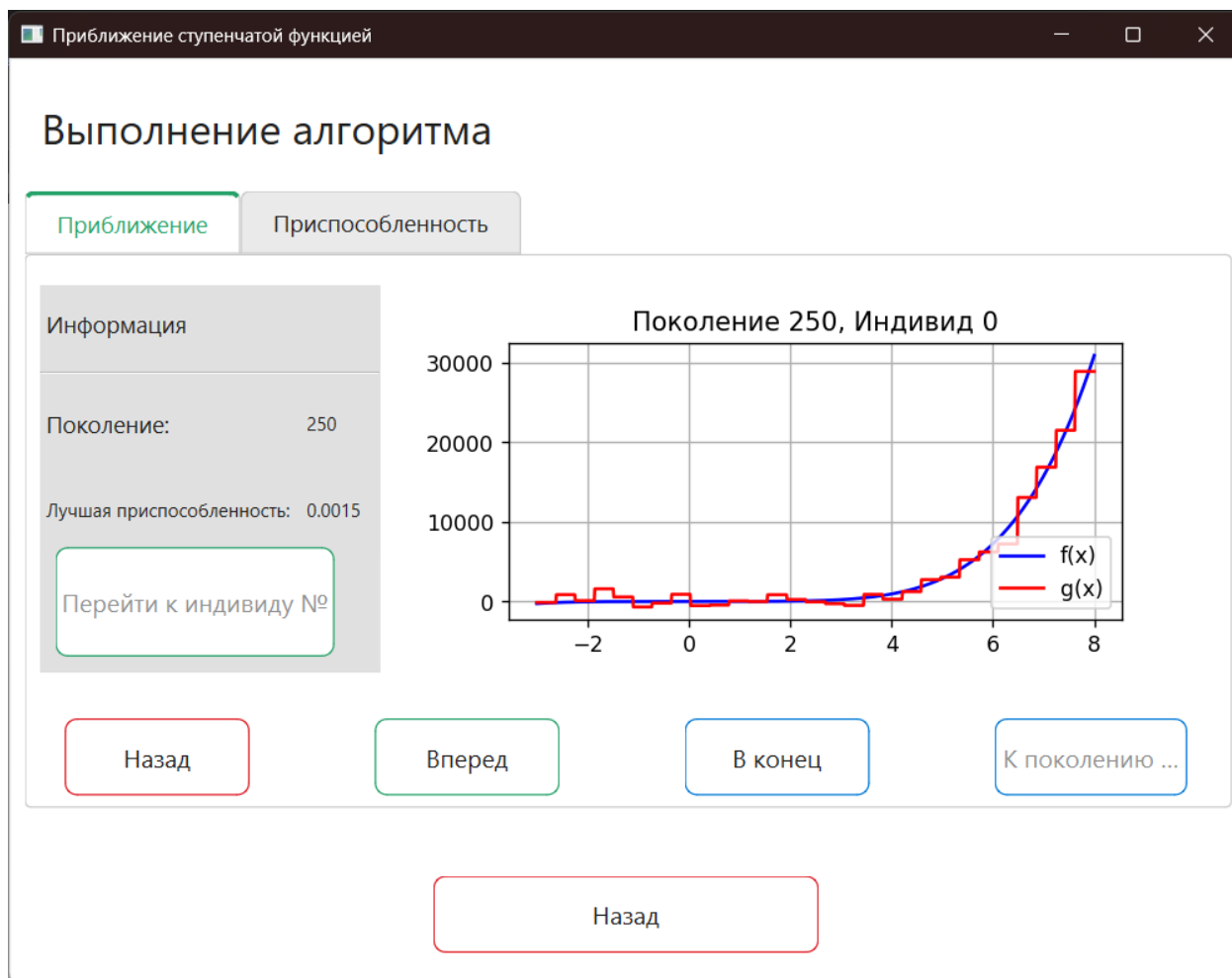


Рисунок 5 — Окно выполнение алгоритма. Приближение



Рисунок 6 — Окно выполнения алгоритма. Приспособленность